

GUÍA TÉCNICA DE ARQUITECTURA: SIMO CO₂

Ingeniería Civil en Informática | Prototipo Funcional Avance 2

1. Introducción y Propósito

El sistema **SIMO CO₂** está diseñado para la monitorización en tiempo real de huella de carbono. Esta guía detalla las decisiones de diseño que garantizan la integridad de los datos y la escalabilidad del software bajo estándares de ingeniería.

2. Arquitectura de Software y Patrones GoF

Se ha implementado una arquitectura orientada a objetos (POO) utilizando **TypeScript** para asegurar un tipado fuerte y contratos claros mediante interfaces.

A. Patrón Observer (Reactividad)

Ubicación: `src/distributor/DataDistributor.ts`

- **Sujeto:** `DataDistributor` centraliza las notificaciones.
- **Observadores:** Clases `GraficoLinea` y `GraficoCircular`.
- **Justificación:** Permite añadir n-visualizadores sin modificar la lógica de captura de los sensores.

B. Patrón Strategy (Flexibilidad de Cálculo)

Ubicación: `src/calculo/`

- **Contexto:** El sistema selecciona dinámicamente el factor de conversión.
- **Estrategias:** `NormativaChile` (factor 0.5) y `NormativaEU` (factor 0.3).
- **Justificación:** Cumple con el principio *Open/Closed*; el sistema está abierto a nuevas leyes ambientales sin alterar el núcleo.

C. Patrón Singleton (Consistencia)

Ubicación: `src/repository/Repositorio.ts`

- **Implementación:** Instancia única global mediante `getInstance()`.
- **Justificación:** Evita duplicidad de datos en memoria y asegura que todos los widgets consulten el mismo estado de sensores.

3. Estructura del Prototipo Funcional

Carpeta/Módulo	Responsabilidad Técnica	Concepto Clave
/sensors	Simulación de hardware y generación de señales base.	Abstracción
/interfaces	Definición de contratos (Widget, Sensor, Strategy).	Polimorfismo
/widgets	Renderización de datos procesados (Observer pattern).	Desacoplamiento
/usuarios	Control de acceso y segregación de perfiles.	Encapsulamiento

4. Justificación de Herramientas

- **TypeScript:** Elegido por su capacidad de detectar errores en tiempo de compilación, vital para sistemas de monitorización crítica.
- **Node.js:** Entorno de ejecución asíncrono que facilita el flujo continuo de datos de sensores mediante `setInterval`.

5. Guía de Ejecución

Para iniciar el sistema de pruebas:

```
npm install
npx ts-node src/index.ts
```